

SmartHealth Clinic management System

INFO6002 DATABASE MANAGEMENT 2

JOHN FINNEY JAWAHAR, YOON SEONG LEE, SHEHERYAR AZAM

Table of Contents

1. Assumptions.....	2
1.1 General Design Assumptions.....	2
1.2 Constraint and Implementation Assumptions	2
2. EER Model	3
2.1 EER Class Diagram.....	3
2.2 Entity and Attribute Summary.....	4
Strong entities.....	4
Subtypes of Staff.....	4
Dependent (existence-dependent) entities.....	4
Associative entities	5
2.3 Relationship and Multiplicity Summary	5
2.4 Type Hierarchy: Staff, Doctor, Nurse, Receptionist	6
2.5 Weak / Dependent Entities	6
2.6 Associative Entities and M:N Relationships	7
3. Relational Model.....	7
3.1 Mapping from EER to Relations	7
3.2 Functional Dependencies and BCNF Justification	13
3.3 Final Normalised DBDL Schema	20
Clinic	20
3.4 Nullability, Referential Actions and CHECK Constraints Summary	23

1. Assumptions

1.1 General Design Assumptions

1. Each staff member works at exactly one clinic (the scenario states a clinic employs many staff but does not mention staff working across clinics).
2. Staff specialization is total and disjoint; every staff member is exactly one of Doctor, Nurse, or Receptionist.
3. The role attribute is used as the discriminator for the Staff type hierarchy.
4. PatientProfile is existence-dependent on Patient via a mandatory 1:1 relationship.
5. AppointmentRecord is existence-dependent on Appointment.
6. preferredClinic and preferredDoctor are optional relationships, not stored attributes.
7. Nurse participation in an appointment is optional.
8. An appointment may be created without a confirmed medical service; a completed appointment is expected to include at least one service.
9. operatingHours is stored as a simple attribute (per-day modelling is not required).

1.2 Constraint and Implementation Assumptions

- {PK = clinicID + roomNo} — ConsultationRoom is identified within a clinic only.
- No double-booking: the same doctor, nurse, or consultation room cannot be assigned to overlapping confirmed appointments.
- A completed appointment must have exactly one invoice; cancelled / no-show appointments may have an invoice only if a penaltyCharge applies.
- A discount approver must be a Doctor or a Receptionist. The SQL CHECK constraint ensures that an approver is recorded when discountAmount > 0; the role restriction is treated as a business/application rule, or could be enforced by a trigger.
- Clinic phoneNumber, Staff email, Patient medicareOrInsuranceNo, and Doctor providerNumber are assumed to be unique and are therefore treated as alternate keys.
- Clinic-specific service price and currency are stored for each (clinicID, serviceID) combination in ClinicService. Only one current clinic-service price record is stored for each combination, so multiple historical price versions for the same clinic-service pair are not modelled.

- finalAmountPayable is treated as a derived value calculated from consultationCharges, prescriptionCharges, penaltyCharge, and discountAmount. It is removed as an independently stored attribute during normalisation and implemented as a computed column in the SQL schema.
- Each appointment is treated as a fixed 30-minute time slot for double-booking checks in the stored procedure and ORM booking logic.
- A service can be added to a booking only if that service is offered by the selected clinic through ClinicService.

2. EER Model

2.1 EER Class Diagram

The EER class diagram is shown below as Figure A1, with a full-size landscape version provided in Appendix A for readability. It uses UML notation throughout: open-triangle generalisation for the Staff hierarchy, filled-diamond composition for the three existence-dependent entities, UML multiplicities (1, 0..1, 1..*, 0..*), and dog-eared notes for constraints that cannot be shown with multiplicity alone. The same content is given in precise tabular form in Sections 2–3.

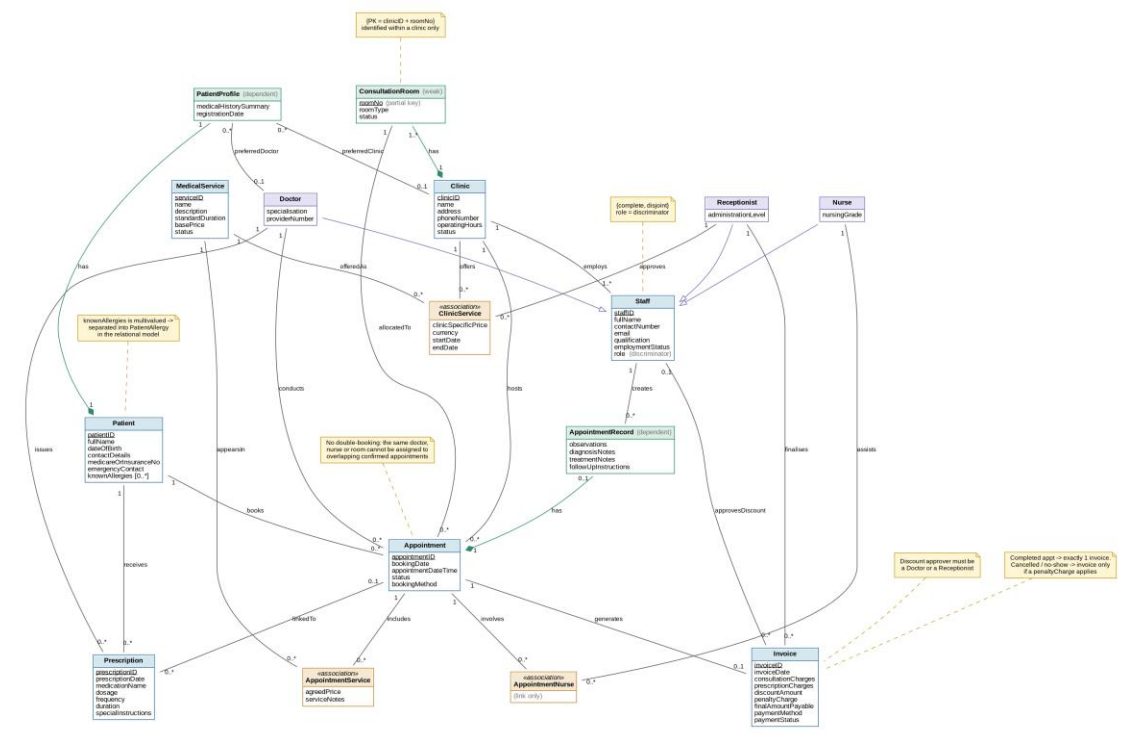


Figure A1. SmartHealth EER Class Diagram (UML)

2.2 Entity and Attribute Summary

Strong entities

Entity	Primary key	Other attributes
Clinic	clinicID	name, address, phoneNumber, operatingHours, status
Staff	staffID	fullName, contactNumber, email, qualification, employmentStatus, role (discriminator)
Patient	patientID	fullName, dateOfBirth, contactDetails, medicareOrInsuranceNo, emergencyContact, knownAllergies [0..*]
MedicalService	serviceID	name, description, standardDuration, basePrice, status
Appointment	appointmentID	bookingDate, appointmentDateTime, status, bookingMethod
Prescription	prescriptionID	prescriptionDate, medicationName, dosage, frequency, duration, specialInstructions
Invoice	invoiceID	invoiceDate, consultationCharges, prescriptionCharges, discountAmount, penaltyCharge, finalAmountPayable(derived — implemented as a computed column), paymentMethod, paymentStatus

Subtypes of Staff

Subtype	Specific attributes
Doctor	specialization, providerNumber
Nurse	nursingGrade
Receptionist	administrationLevel

Dependent (existence-dependent) entities

Entity	Identifier	Other attributes	Owner
ConsultationRoom	(clinicID, roomNo)	roomType, status	Clinic (composition)
PatientProfile	patientID	medicalHistorySummary, registrationDate	Patient (composition, 1:1)
AppointmentRecord	appointmentID	observations, diagnosisNotes, treatmentNotes, followUpInstructions; createdBy → Staff	Appointment (composition)

Associative entities

Entity	Resolves M:N between	Own attributes
ClinicService	Clinic ↔ MedicalService	clinicSpecificPrice, currency, startDate, endDate; approved by a Receptionist
AppointmentService	Appointment ↔ MedicalService	agreedPrice, serviceNotes
AppointmentNurse	Appointment ↔ Nurse	(pure link — no own attributes)

2.3 Relationship and Multiplicity Summary

Multiplicities are stated in plain English to avoid any ambiguity when read away from the diagram.

Relationship	Multiplicity (reading)
Clinic employs Staff	Each clinic employs 1..* staff; each staff belongs to exactly 1 clinic.
Clinic has ConsultationRoom (composition)	Each clinic has 1..* rooms; each room belongs to exactly 1 clinic.
Patient has PatientProfile (composition)	Each patient has exactly 1 profile; each profile belongs to exactly 1 patient.
PatientProfile preferredClinic Clinic	Each profile may name 0..1 preferred clinic; a clinic may be preferred by 0..* profiles.
PatientProfile preferredDoctor Doctor	Each profile may name 0..1 preferred doctor; a doctor may be preferred by 0..* profiles.
Clinic offers MedicalService (via ClinicService)	Each clinic offers 0..* services; each service is offered by 0..* clinics (M:N).
Receptionist approves ClinicService	Each offering is approved by one receptionist when created; if the receptionist record is later removed, the approval reference may be set to NULL. A receptionist may approve 0..* offerings.
Patient books Appointment	Each patient books 0..* appointments; each appointment is booked by exactly 1 patient.
Clinic hosts Appointment	Each clinic hosts 0..* appointments; each appointment occurs at exactly 1 clinic.
Doctor conducts Appointment	Each appointment is conducted by exactly 1 doctor; a doctor conducts 0..* appointments.
ConsultationRoom allocatedTo Appointment	Each appointment is allocated exactly 1 room; a room serves 0..* appointments.
Appointment involves Nurse (via AppointmentNurse)	An appointment involves 0..* nurses; a nurse assists 0..* appointments (M:N, optional).

Relationship	Multiplicity (reading)
Appointment includes MedicalService (via AppointmentService)	An appointment includes 0..* services; a service appears in 0..* appointments (M:N).
Appointment has AppointmentRecord (composition)	Each appointment has 0..1 record; each record belongs to exactly 1 appointment.
Staff creates AppointmentRecord	Each record is created by exactly 1 staff member; a staff member creates 0..* records.
Doctor issues Prescription	Each prescription is issued by exactly 1 doctor; a doctor issues 0..* prescriptions.
Patient receives Prescription	Each prescription belongs to exactly 1 patient; a patient receives 0..* prescriptions.
Appointment linkedTo Prescription	Each appointment may have 0..* prescriptions; each prescription may be linked to 0..1 appointment.
Appointment generates Invoice	Each appointment generates 0..1 invoice; each invoice belongs to exactly 1 appointment.
Receptionist finalises Invoice	Each invoice is finalised by exactly 1 receptionist; a receptionist finalises 0..* invoices.
Staff approvesDiscount Invoice	An invoice discount is approved by 0..1 staff (Doctor or Receptionist); a staff member approves 0..* discounts.

2.4 Type Hierarchy: Staff, Doctor, Nurse, Receptionist

Staff is generalised into three subtypes: Doctor, Nurse, and Receptionist. The scenario describes exactly these three staff types, each with its own attributes (Doctor: specialization, providerNumber; Nurse: nursingGrade; Receptionist: administrationLevel), which is the classic signal for a specialization hierarchy.

The specialization is modelled as **{complete, disjoint}**: every staff member is exactly one of the three types (complete/total) and cannot be more than one at the same time (disjoint). The existing *role* attribute on Staff is used as the discriminator for this hierarchy, so it is not duplicated information.

Note: the subtype-specific relationships connect to the subtypes, not to the Staff supertype — a Doctor conducts appointments and issues prescriptions, a Nurse assists appointments, and a Receptionist finalises invoices and approves clinic-service offerings. Only the generic “creates AppointmentRecord” relationship connects to Staff as a whole.

2.5 Weak / Dependent Entities

- ConsultationRoom is a weak entity. Its room number is unique only within a clinic (the scenario notes two clinics may both have a Room 101), so it cannot be

identified on its own. It is identified by the composite key (clinicID, roomNo) and modelled as a composition of Clinic.

- PatientProfile is existence-dependent on Patient. The scenario states a profile cannot exist without its patient, and each patient has exactly one profile. It is modelled as a 1:1 composition; its primary key is patientID.
- AppointmentRecord is existence-dependent on Appointment. Each appointment has at most one record, and a record cannot exist without its appointment. It is modelled as a composition; its primary key is appointmentID. The creating staff member is captured by a separate “creates” relationship.

2.6 Associative Entities and M:N Relationships

The checklist allows many-to-many relationships to be modelled with associative entities provided a justification is given. Three M:N relationships carry their own attributes, which cannot be stored on a plain relationship line, so each is promoted to an associative entity:

- ClinicService — Clinic and MedicalService are M:N, and each clinic-service combination stores its own clinic-specific price, currency, start date, end date, and approving receptionist.
- AppointmentService — Appointment and MedicalService are M:N, and each combination stores the agreed price and service notes.
- AppointmentNurse — Appointment and Nurse are M:N. It carries no own attributes (a pure link) but is still required to resolve the many-to-many relationship; nurse participation is optional.

3. Relational Model

3.1 Mapping from EER to Relations

Clinic

Clinic(clinicID, name, address, phoneNumber, operatingHours, status)

Primary Key clinicID

This is a strong entity and it has been directed mapped using simple attributes. The primary key for this relation is chosen as clinicID.

ConsultationRoom

ConsultationRoom(clinicID, roomNo, roomType, status)

Primary Key clinicID, roomNo

Foreign Key clinicID references Clinic(clinicID)

ON UPDATE NO ACTION, ON DELETE CASCADE

A weak entity which depends on Clinic. The composite primary key in this relation is (clinicID, roomNo).

Staff

Staff(staffID, fullName, contactNumber, email, qualification, employmentStatus, role, clinicID)

Primary Key staffID

Foreign Key clinicID references Clinic(clinicID)

ON UPDATE NO ACTION, ON DELETE NO ACTION

This is the parent entity for Doctor, Receptionist and Nurse subclasses. Here, the role attribute is considered as the discriminator.

Doctor

Doctor(staffID, specialization, providerNumber)

Primary Key staffID

Foreign Key staffID references Staff(staffID)

ON UPDATE NO ACTION, ON DELETE CASCADE

Doctor is a subclass of the Staff relation. It contains the staffID as foreign key which references the primary key from the parent entity.

Receptionist

Receptionist(staffID, administrationLevel)

Primary Key staffID

Foreign Key staffID references Staff(staffID)

ON UPDATE NO ACTION, ON DELETE CASCADE

Receptionist is the second subclass of the Staff relation. It contains the staffID as foreign key which references the primary key from the parent entity.

Nurse

Nurse(staffID, nursingGrade)

Primary Key staffID

Foreign Key staffID references Staff(staffID)

ON UPDATE NO ACTION, ON DELETE CASCADE

Nurse is the last subclass of the Staff relation. It also contains the staffID as foreign key which references the primary key from the parent entity.

Patient

Patient(patientID, firstName, middleName, lastName, dateOfBirth, contactDetails, medicareOrInsuranceNo, emergencyContact)

Primary Key patientID

Patient is a strong entity that has been directly mapped to its corresponding relation. The composite attribute name has been decomposed into the firstName, middleName and lastName simple attributes. In this relation, knownAllergies is also a multi-valued attribute. Hence, a new relation PatientAllergy has been created.

PatientAllergy

PatientAllergy(patientID, allergyName)

Primary Key (patientID, allergyName)

Foreign Key patientID references Patient(patientID)

ON UPDATE NO ACTION, ON DELETE CASCADE

The multi-valued attribute knownAllergies from Patient is represented in this relation. Here, the composite primary key (patientID, allergyName) is used to uniquely identify each record. Additionally, the foreign key patientID references the parent entity Patient(patientID).

PatientProfile

PatientProfile(patientID, medicalHistorySummary, preferredClinicID, preferredDoctorID, registrationDate)

Primary Key patientID

Foreign Key patientID references Patient(patientID)

ON UPDATE NO ACTION, ON DELETE CASCADE

Foreign Key preferredClinicID references Clinic(clinicID)

ON UPDATE NO ACTION, ON DELETE SET NULL

Foreign Key preferredDoctorID references Doctor(staffID)

ON UPDATE NO ACTION, ON DELETE SET NULL

PatientProfile is a weak entity that is dependent on Patient. Its primary key patientID references the value from the Patient relation.

MedicalService

MedicalService(serviceID, name, description, standardDuration, basePrice, status)

Primary Key serviceID

There are no dependencies for MedicalService. Hence, it's a strong entity that has been directly converted into its relation.

ClinicService

ClinicService(clinicID, serviceID, clinicSpecificPrice, currency, startDate, endDate, approvingReceptionistID)

Primary Key clinicID, serviceID

Foreign Key clinicID references Clinic(clinicID)

ON UPDATE NO ACTION, ON DELETE CASCADE

Foreign Key serviceID references MedicalService(serviceID)

ON UPDATE NO ACTION, ON DELETE CASCADE

Foreign Key approvingReceptionistID references Receptionist(staffID)

ON UPDATE NO ACTION, ON DELETE SET NULL

This entity is formed from the Clinic and MedicalService relations. For this relation, both clinicID and serviceID are the primary keys.

Appointment

Appointment(appointmentID, bookingDate, appointmentDateTime, status, bookingMethod, patientID, doctorID, clinicID, roomNo)

Primary Key appointmentID

Foreign Key patientID references Patient(patientID)

ON UPDATE NO ACTION, ON DELETE CASCADE

Foreign Key doctorID references Doctor(staffID)

ON UPDATE NO ACTION, ON DELETE CASCADE

Foreign Key (clinicID, roomNo) references ConsultationRoom(clinicID, roomNo)

ON UPDATE NO ACTION, ON DELETE CASCADE

Appointment is a strong entity with foreign keys referencing Patient, Doctor, and ConsultationRoom. Here, the primary key is the appointmentID attribute.

AppointmentRecord

AppointmentRecord(appointmentID, observations, diagnosisNotes, treatmentNotes, followUpInstructions, staffID)

Primary Key appointmentID

Foreign Key appointmentID references Appointment(appointmentID)

ON UPDATE NO ACTION, ON DELETE CASCADE

Foreign Key staffID references Staff(staffID)

ON UPDATE NO ACTION, ON DELETE NO ACTION

This is a weak entity which uses the primary key column from the parent entity. The AppointmentRecord relation has a 1:1 relationship with the parent Appointment entity.

AppointmentService

AppointmentService(appointmentID, serviceID, agreedPrice, serviceNotes)

Primary Key appointmentID, serviceID

Foreign Key appointmentID references Appointment(appointmentID)

ON UPDATE NO ACTION, ON DELETE CASCADE

Foreign Key serviceID references MedicalService(serviceID)

ON UPDATE NO ACTION, ON DELETE CASCADE

It's formed from the Appointment and Service relations. appointmentID and serviceID are both primary keys in the AppointmentService relation.

AppointmentNurse

AppointmentNurse(appointmentID, staffID)

Primary Key appointmentID, staffID

Foreign Key appointmentID references Appointment(appointmentID)

ON UPDATE NO ACTION, ON DELETE CASCADE

Foreign Key staffID references Nurse(staffID)

ON UPDATE NO ACTION, ON DELETE NO ACTION

The subclass Nurse contains staffID from the parent entity. This primary key is combined along with the appointmentID in AppointmentNurse to form this relation.

Invoice

Invoice(invoiceID, invoiceDate, consultationCharges, prescriptionCharges, discountAmount, penaltyCharge, finalAmountPayable, paymentMethod, paymentStatus, appointmentID, finalisedByReceptionistID, discountApprovedByStaffID)

Primary Key invoiceID

Unique (appointmentID)

Foreign Key appointmentID references Appointment(appointmentID)

ON UPDATE NO ACTION, ON DELETE CASCADE

Foreign Key finalisedByReceptionistID references Receptionist(staffID)

ON UPDATE NO ACTION, ON DELETE NO ACTION

Foreign Key discountApprovedByStaffID references Staff(staffID)

ON UPDATE NO ACTION, ON DELETE NO ACTION

Invoice is a strong entity derived directly from the EER. While appointmentID is a unique key, finalisedByReceptionistID and discountApprovedByStaffID are foreign keys in this relation. The primary key in this relation is invoiceID.

Prescription

Prescription(prescriptionID, prescriptionDate, medicationName, dosage, frequency, duration, specialInstructions, appointmentID, patientID, doctorID)

Primary Key prescriptionID

Foreign Key appointmentID references Appointment(appointmentID) (nullable — appointment link is optional)

ON UPDATE NO ACTION, ON DELETE SET NULL

Foreign Key patientID references Patient(patientID)

ON UPDATE NO ACTION, ON DELETE NO ACTION

Foreign Key doctorID references Doctor(staffID)

ON UPDATE NO ACTION, ON DELETE NO ACTION

This relation is a strong entity which contains the primary key prescriptionID. Additionally, foreign keys such as appointmentID, patientID and doctorID are also used to form this relation.

3.2 Functional Dependencies and BCNF Justification

Clinic

Clinic (clinicID, name, address, phoneNumber, operatingHours, status)

This is a strong entity, and it has been directed mapped using simple attributes. The primary key for this relation is chosen as clinicID.

FD: clinicID $\rightarrow$ name, address, phoneNumber, operatingHours, status

FD: phoneNumber $\rightarrow$ clinicID

This relation is in BCNF because every FD has a candidate key on the left side. clinicID determines all other attributes while phoneNumber is an Alternate key, as we assume each clinic has a unique line determines the rest. The key is a single attribute, so no partial dependency is possible, and there are no transitive dependencies.

Consultation Room

ConsultationRoom (clinicID, roomNo, roomType, status)

FD: clinicID, roomNo $\rightarrow$ roomType, status

This relation is in BCNF because the only functional dependency has the composite key (clinicID, roomNo) on the left side. roomType and status are functionally dependent on the whole key, not on clinicID or roomNo alone, so there is no partial dependency, and there are no transitive dependencies.

Staff

Staff (staffID, fullName, contactNumber, email, qualification, employmentStatus, role, clinicID)

This is the parent entity for Doctor, Receptionist and Nurse subclasses. Here, the role attribute is considered as the discriminator.

FD: staffID $\rightarrow$ fullName, contactNumber, email, qualification, employmentStatus, role, clinicID

FD: email $\rightarrow$ staffID

This relation is in BCNF because both functional dependencies have a candidate key on the left side. staffID is the primary key and email is an alternate key, as we assume email is unique per staff member. All non-key attributes are functionally dependent on staffID. Nothing is functionally dependent on the attribute role, which is the subtype discriminator or on clinicID, so there are no transitive dependencies, and with a single attribute key there are no partial dependencies.

Doctor

Doctor (staffID, specialization, providerNumber)

Doctor is a subclass of the Staff relation. It contains the staffID as foreign key which references the primary key from the parent entity.

FD: staffID $\rightarrow$ specialization, providerNumber

FD: providerNumber $\rightarrow$ staffID

This relation is in BCNF because both functional dependencies have a candidate key on the left side. staffID is the primary key and providerNumber an alternate key, since we assume provider numbers are unique. specialization and providerNumber are functionally dependent on staffID, with no partial dependency and no transitive dependency.

Receptionist

Receptionist (staffID, administrationLevel)

Receptionist is the second subclass of the Staff relation. It contains the staffID as foreign key which references the primary key from the parent entity.

FD: staffID $\rightarrow$ administrationLevel

This relation is in BCNF because the only functional dependency has staffID which is the primary key on the left side. administrationLevel is functionally dependent on staffID, with no transitive dependency.

Nurse

Nurse (staffID, nursingGrade)

Nurse is the last subclass of the Staff relation. It also contains the staffID as foreign key which references the primary key from the parent entity.

FD: staffID $\rightarrow$ nursingGrade

This relation is in BCNF because the only functional dependency has staffID which is the primary key on the left side. nursingGrade is functionally dependent on staffID, with no transitive dependency.

Patient

Patient (patientID, firstName, middleName, lastName, dateOfBirth, contactDetails, medicareOrInsuranceNo, emergencyContact)

Patient is a strong entity that has been directly mapped to its corresponding relation. The composite attribute name has been decomposed into the firstName, middleName and lastName simple attributes. In this relation, knownAllergies is also a multi-valued attribute. Hence, a new relation PatientAllergy has been created.

FD: patientID $\rightarrow$ firstName, middleName, lastName, dateOfBirth, contactDetails, medicareOrInsuranceNo, emergencyContact

FD: medicareOrInsuranceNo $\rightarrow$ patientID

This relation is in BCNF because both functional dependencies have a candidate key on the left side. patientID is the primary key and medicareOrInsuranceNo an alternate key, since we assume they are unique per patient. All other attributes are functionally dependent on patientID. The composite attribute name has been split into the atomic attributes firstName, middleName and lastName (satisfying 1NF), and the multi-valued attribute knownAllergies has been moved to its own relation (PatientAllergy below), so no non-atomic or repeating attribute remains. With a single-attribute key there is no partial dependency, and there are no transitive dependencies.

PatientAllergy

PatientAllergy (patientID, allergyName)

The multi-valued attribute knownAllergies from Patient is represented in this relation. Here, the composite primary key (patientID, allergyName) is used to uniquely identify each record. Additionally, the foreign key patientID references the parent entity Patient(patientID).

This relation is the result of removing the multi-valued attribute knownAllergies from Patient. It has no non-key attributes, and its only candidate key is the whole key (patientID, allergyName). Because there is no non-trivial functional dependency, it is automatically in BCNF. An all-key relation cannot have a non-key attribute on either side of a dependency, so no partial or transitive dependency is possible.

PatientProfile

PatientProfile(patientID, medicalHistorySummary, preferredClinicID, preferredDoctorID, registrationDate)

PatientProfile is a weak entity that is dependent on Patient. Its primary key patientID references the value from the Patient relation.

FD: patientID $\rightarrow$ medicalHistorySummary, preferredClinicID, preferredDoctorID, registrationDate

This relation is in BCNF because the only functional dependency has patientID which is the primary key on the left side. medicalHistorySummary, preferredClinicID, preferredDoctorID and registrationDate are functionally dependent on patientID. preferredClinicID and preferredDoctorID are foreign-key references only and do not have any other attribute functionally dependent on them within this relation, so there are no transitive dependencies, and with a single-attribute key there is no partial dependency.

MedicalService

MedicalService (serviceID, name, description, standardDuration, basePrice, status)

There are no dependencies for MedicalService. Hence, it's a strong entity that has been directly converted into its relation.

FD: serviceID $\rightarrow$ name, description, standardDuration, basePrice, status

This relation is in BCNF because the only functional dependency has serviceID that is the primary key on the left side. name, description, standardDuration, basePrice and status are functionally dependent on serviceID, with no partial dependency due to a single attribute key and no transitive dependency.

ClinicService

ClinicService(clinicID, serviceID, clinicSpecificPrice, currency, startDate, endDate, approvingReceptionistID)

This entity is formed from the Clinic and MedicalService relations. For this relation, both clinicID and serviceID are the primary keys.

FD: clinicID, serviceID $\rightarrow$ clinicSpecificPrice, currency, startDate, endDate, approvingReceptionistID

This relation resolves the many-to-many relationship between Clinic and MedicalService. It is in BCNF because the only functional dependency has the full composite key (clinicID, serviceID) on the left side. clinicSpecificPrice, currency, startDate, endDate and approvingReceptionistID are functionally dependent on the whole key, and no single part of the key functionally determines any of them. Currency is stored per clinic-service combination as stated in the scenario, so we do not assume clinicID currency. There is therefore no partial dependency and no transitive dependency.

Appointment

Appointment (appointmentID, bookingDate, appointmentDateTime, status, bookingMethod, patientID, doctorID, clinicID, roomNo)

Appointment is a strong entity with foreign keys referencing Patient, Doctor, and ConsultationRoom. Here, the primary key is the appointmentID attribute.

FD: appointmentID $\rightarrow$ bookingDate, appointmentDateTime, status, bookingMethod, patientID, doctorID, clinicID, roomNo

This relation is in BCNF because the only functional dependency has appointmentID that is the Primary key on the left side, and every other attribute is functionally dependent on appointmentID. With a single-attribute key there is no partial dependency. There is also no transitive dependency.

clinicID is not functionally dependent on doctorID, because clinicID records where the appointment was held (fixed at booking) while the doctor's employment clinic is stored separately in Staff and can change over time, so the same doctor's appointments may have different clinics.

AppointmentRecord

AppointmentRecord (appointmentID, observations, diagnosisNotes, treatmentNotes, followUpInstructions, staffID)

This is a weak entity which uses the primary key column from the parent entity. The AppointmentRecord relation has a 1:1 relationship with the parent Appointment entity.

FD: appointmentID $\rightarrow$ observations, diagnosisNotes, treatmentNotes, followUpInstructions, staffID

This relation is in BCNF because the only functional dependency has appointmentID that is the Primary key on the left side. The record is dependent on the existence of appointment (1:1), so appointmentID serves as both Primary key and Foreign key. observations, diagnosisNotes, treatmentNotes, followUpInstructions and staffID are functionally dependent on appointmentID. No other attribute in the record is functionally dependent on staffID (the creating staff member), so there is no transitive dependency, and with a single-attribute key there is no partial dependency.

AppointmentService

AppointmentService (appointmentID, serviceID, agreedPrice, serviceNotes)

It's formed from the Appointment and Service relations. appointmentID and serviceID are both primary keys in the AppointmentService relation.

FD: appointmentID, serviceID $\rightarrow$ agreedPrice, serviceNotes

This relation resolves the many-to-many relationship between Appointment and MedicalService. It is in BCNF because the only functional dependency has the full composite key (appointmentID, serviceID) on the left side. agreedPrice and serviceNotes are functionally dependent on the whole key. Neither is fixed by appointmentID alone or serviceID alone (the agreed price is negotiated for that service within that specific appointment, so it is not fixed by the service's base price), so there is no partial dependency and no transitive dependency.

AppointmentNurse

AppointmentNurse(appointmentID, staffID)

The subclass Nurse contains staffID from the parent entity. This primary key is combined along with the appointmentID in AppointmentNurse to form this relation.

FD: none other than the trivial dependency of the key on itself

This relation resolves the optional many-to-many relationship between Appointment and Nurse and carries no attributes of its own. Its only candidate key is the whole key (appointmentID, staffID), and because there is no non-trivial functional dependency, it is automatically in BCNF, no partial or transitive dependency is possible in an all-key relation.

Invoice

Invoice (invoiceID, invoiceDate, consultationCharges, prescriptionCharges, discountAmount, penaltyCharge, finalAmountPayable, paymentMethod, paymentStatus, appointmentID, finalisedByReceptionistID, discountApprovedByStaffID)

Invoice is a strong entity derived directly from the EER. While appointmentID is a unique key, finalisedByReceptionistID and discountApprovedByStaffID are foreign keys in this relation. The primary key in this relation is invoiceID.

FD1: invoiceID $\rightarrow$ invoiceDate, consultationCharges, prescriptionCharges, discountAmount, penaltyCharge, finalAmountPayable, paymentMethod, paymentStatus, appointmentID, finalisedByReceptionistID, discountApprovedByStaffID

FD2: appointmentID $\rightarrow$ invoiceID (each appointment has at most one invoice, so appointmentID is an alternate key)

FD3: consultationCharges, prescriptionCharges, penaltyCharge, discountAmount $\rightarrow$ finalAmountPayable

In FD3 finalAmountPayable is functionally dependent on the combination of consultationCharges, prescriptionCharges, penaltyCharge and discountAmount. This is not in BCNF form as a non-key attribute (finalAmountPayable) is functionally dependent on a set of non-key attributes, this is a transitive dependency which violates the BCNF form.

After identifying the violation, we can see that finalAmountPayable is a derived attribute. A stored copy of a value that can always be recomputed from the four charge attributes already in the relation. So we can resolve it by removing finalAmountPayable from the stored relation. This is lossless because the value is fully recoverable by computation from the retained attributes. At implementation it can be provided as a computed column or through a view. After removal, FD1 no longer includes finalAmountPayable. The remaining dependencies are invoiceID $\rightarrow$ invoiceDate, consultationCharges, prescriptionCharges, discountAmount, penaltyCharge, paymentMethod, paymentStatus, appointmentID, finalisedByReceptionistID, discountApprovedByStaffID, and appointmentID $\rightarrow$ invoiceID. Both determinants are candidate keys, so the final Invoice relation is in BCNF.

Prescription

Prescription (prescriptionID, prescriptionDate, medicationName, dosage, frequency, duration, specialInstructions, appointmentID, patientID, doctorID)

This relation is a strong entity which contains the primary key prescriptionID. Additionally, foreign keys such as appointmentID, patientID and doctorID are also used to form this relation.

FD: prescriptionID $\rightarrow$ prescriptionDate, medicationName, dosage, frequency, duration, specialInstructions, appointmentID, patientID, doctorID

This relation is in BCNF because the only functional dependency has the primary key prescriptionID on the left side, and every other attribute is functionally dependent on

prescriptionID. With a single-attribute key there is no partial dependency. There is no transitive dependency: patientID (the patient the prescription is for) and doctorID (the issuing doctor) are recorded directly on the prescription as mandatory facts, and the appointment link (appointmentID) is optional, so neither patientID nor doctorID is functionally dependent on appointmentID.

3.3 Final Normalised DBDL Schema

Clinic

Clinic(clinicID, name, address, phoneNumber, operatingHours, status)

Primary Key clinicID

Alternate Key phoneNumber

ConsultationRoom

ConsultationRoom(clinicID, roomNo, roomType, status)

Primary Key clinicID, roomNo

Foreign Key clinicID references Clinic(clinicID)

ON UPDATE NO ACTION, ON DELETE CASCADE

Staff

Staff(staffID, fullName, contactNumber, email, qualification, employmentStatus, role, clinicID)

Primary Key staffID

Alternate Key email

Foreign Key clinicID references Clinic(clinicID)

ON UPDATE NO ACTION, ON DELETE NO ACTION

Doctor

Doctor(staffID, specialization, providerNumber)

Primary Key staffID

Alternate Key providerNumber

Foreign Key staffID references Staff(staffID)

ON UPDATE NO ACTION, ON DELETE CASCADE

Receptionist

Receptionist(staffID, administrationLevel)

Primary Key staffID

Foreign Key staffID references Staff(staffID)

ON UPDATE NO ACTION, ON DELETE CASCADE

Nurse

Nurse(staffID, nursingGrade)
Primary Key staffID
Foreign Key staffID references Staff(staffID)
ON UPDATE NO ACTION, ON DELETE CASCADE

Patient

Patient(patientID, firstName, middleName, lastName, dateOfBirth, contactDetails, medicareOrInsuranceNo, emergencyContact)
Primary Key patientID
Alternate Key medicareOrInsuranceNo

PatientAllergy

PatientAllergy(patientID, allergyName)
Primary Key patientID, allergyName
Foreign Key patientID references Patient(patientID)
ON UPDATE NO ACTION, ON DELETE CASCADE

PatientProfile

PatientProfile(patientID, medicalHistorySummary, preferredClinicID, preferredDoctorID, registrationDate)
Primary Key patientID
Foreign Key patientID references Patient(patientID)
ON UPDATE NO ACTION, ON DELETE CASCADE
Foreign Key preferredClinicID references Clinic(clinicID)
ON UPDATE NO ACTION, ON DELETE SET NULL
Foreign Key preferredDoctorID references Doctor(staffID)
ON UPDATE NO ACTION, ON DELETE SET NULL

MedicalService

MedicalService(serviceID, name, description, standardDuration, basePrice, status)
Primary Key serviceID

ClinicService

ClinicService(clinicID, serviceID, clinicSpecificPrice, currency, startDate, endDate, approvingReceptionistID)
Primary Key clinicID, serviceID
Foreign Key clinicID references Clinic(clinicID)
ON UPDATE NO ACTION, ON DELETE CASCADE
Foreign Key serviceID references MedicalService(serviceID)
ON UPDATE NO ACTION, ON DELETE CASCADE
Foreign Key approvingReceptionistID references Receptionist(staffID)
ON UPDATE NO ACTION, ON DELETE SET NULL

Appointment

Appointment(appointmentID, bookingDate, appointmentDateTime, status, bookingMethod, patientID, doctorID, clinicID, roomNo)
Primary Key appointmentID
Foreign Key patientID NOT NULL references Patient(patientID)
ON UPDATE NO ACTION, ON DELETE CASCADE
Foreign Key doctorID NOT NULL references Doctor(staffID)
ON UPDATE NO ACTION, ON DELETE CASCADE
Foreign Key clinicID, roomNo NOT NULL references ConsultationRoom(clinicID, roomNo)
ON UPDATE NO ACTION, ON DELETE CASCADE

AppointmentRecord

AppointmentRecord(appointmentID, observations, diagnosisNotes, treatmentNotes, followUpInstructions, staffID)
Primary Key appointmentID
Foreign Key appointmentID references Appointment(appointmentID)
ON UPDATE NO ACTION, ON DELETE CASCADE
Foreign Key staffID NOT NULL references Staff(staffID)
ON UPDATE NO ACTION, ON DELETE NO ACTION

AppointmentService

AppointmentService(appointmentID, serviceID, agreedPrice, serviceNotes)
Primary Key appointmentID, serviceID
Foreign Key appointmentID references Appointment(appointmentID)
ON UPDATE NO ACTION, ON DELETE CASCADE
Foreign Key serviceID references MedicalService(serviceID)
ON UPDATE NO ACTION, ON DELETE CASCADE

AppointmentNurse

AppointmentNurse(appointmentID, staffID)
Primary Key appointmentID, staffID
Foreign Key appointmentID references Appointment(appointmentID)
ON UPDATE NO ACTION, ON DELETE CASCADE
Foreign Key staffID references Nurse(staffID)
ON UPDATE NO ACTION, ON DELETE NO ACTION

Invoice

Invoice(invoiceID, invoiceDate, consultationCharges, prescriptionCharges, discountAmount, penaltyCharge, paymentMethod, paymentStatus, appointmentID, finalisedByReceptionistID, discountApprovedByStaffID)
Primary Key invoiceID
Alternate Key appointmentID
Foreign Key appointmentID NOT NULL references Appointment(appointmentID)
ON UPDATE NO ACTION, ON DELETE CASCADE
Foreign Key finalisedByReceptionistID NOT NULL references Receptionist(staffID)

ON UPDATE NO ACTION, ON DELETE NO ACTION
 Foreign Key discountApprovedByStaffID references Staff(staffID)
 ON UPDATE NO ACTION, ON DELETE NO ACTION

Prescription

Prescription(prescriptionID, prescriptionDate, medicationName, dosage, frequency, duration, specialInstructions, appointmentID, patientID, doctorID)
 Primary Key prescriptionID
 Foreign Key appointmentID references Appointment(appointmentID)
 ON UPDATE NO ACTION, ON DELETE SET NULL
 Foreign Key patientID NOT NULL references Patient(patientID)
 ON UPDATE NO ACTION, ON DELETE NO ACTION
 Foreign Key doctorID NOT NULL references Doctor(staffID)
 ON UPDATE NO ACTION, ON DELETE NO ACTION

3.4 Nullability, Referential Actions and CHECK Constraints Summary

All foreign keys use **ON UPDATE NO ACTION**. SQL Server rejects ON UPDATE CASCADE wherever a table is reachable by more than one cascade path, which occurs for several relations here, so NO ACTION is applied uniformly on update. The ON DELETE action is chosen per relationship, as summarised below.

Referential actions (ON DELETE)

Action	Where used	Rationale
CASCADE	Composition / existence-dependent children: ConsultationRoom, PatientAllergy, PatientProfile, the Doctor/Nurse/Receptionist subtype rows, Appointment (from Patient, Doctor, Room), AppointmentRecord / AppointmentService / AppointmentNurse / Invoice (from Appointment), ClinicService	The child cannot exist without its parent, so deleting the parent removes the child.
SET NULL	Optional references: PatientProfile.preferredClinicID, PatientProfile.preferredDoctorID, ClinicService.approvingReceptionistID, Prescription.appointmentID	The reference is optional; removing the referenced row clears the link without deleting the referencing row.
NO ACTION	AppointmentRecord.staffID, AppointmentNurse.staffID, Invoice.finalisedByReceptionistID, Invoice.discountApprovedByStaffID, Prescription.patientID, Prescription.doctorID	A CASCADE here would create a second cascade path (rejected by SQL Server), or the

Action	Where used	Rationale
		staff/patient record must be retained for auditing.

Nullability

Columns are NOT NULL only where the business logic requires a value: all primary keys; mandatory foreign keys (e.g. Appointment.patientID / doctorID / (clinicID, roomNo), Prescription.patientID / doctorID, AppointmentRecord.staffID, Invoice.appointmentID / finalisedByReceptionistID); and required descriptive attributes. Optional attributes (e.g. middleName, emergencyContact, preferred clinic / doctor, discountApprovedByStaffID, Prescription.appointmentID, paymentMethod) remain nullable.

CHECK constraints (from createTables.sql)

Constraint	Table	Condition
CK_Clinic_status	Clinic	status IN ('Active','Inactive','Temporarily Closed')
CK_MedicalService_dur	MedicalService	standardDuration > 0
CK_MedicalService_price	MedicalService	basePrice >= 0
CK_MedicalService_statuses	MedicalService	status IN ('Active','Inactive')
CK_Staff_role	Staff	role IN ('Doctor','Nurse','Receptionist')
CK_Staff_emp	Staff	employmentStatus IN ('Full-time','Part-time','Casual','Contract')
CK_ConsultationRoom_status	ConsultationRoom	status IN ('Available','Occupied','Out of Service')
CK_ClinicService_price	ClinicService	clinicSpecificPrice >= 0
CK_ClinicService_dates	ClinicService	endDate IS NULL OR endDate >= startDate
CK_Appointment_status	Appointment	status IN ('Booked','Confirmed','Completed','Cancelled','No-show')
CK_Appointment_method	Appointment	bookingMethod IN ('online','phone','in person')
CK_AppointmentService_price	AppointmentService	agreedPrice >= 0
CK_Invoice_amounts	Invoice	consultationCharges, prescriptionCharges, discountAmount, penaltyCharge all >= 0

Constraint	Table	Condition
CK_Invoice_discAppr	Invoice	discountAmount = 0 OR discountApprovedByStaffID IS NOT NULL
CK_Invoice_payStatus	Invoice	paymentStatus IN ('Unpaid','Paid','Partially Paid')
CK_Invoice_payMethod	Invoice	paymentMethod IS NULL OR paymentMethod IN ('Cash','Card','Medicare','Insurance','Bank Transfer')

Note: finalAmountPayable is removed as an independently stored attribute during normalisation. In the implementation it is provided as a **computed column**, so it cannot be inserted or updated independently and is always recomputed from the four charge columns.

Note: CK_Invoice_discAppr only enforces that a discount approval ID is present when discountAmount > 0. The additional rule that the approver must be a Doctor or Receptionist is treated as a business/application rule, because a standard CHECK constraint cannot validate the subtype of a referenced Staff row.

Appendix A. Full-size EER Class Diagram

